Docker con Spring Boot: multi-stage, volúmenes y redes

Trabajo en Clase 04 — Ingeniería de Software III

Diego López Camacho — Universidad Industrial de Santander

7 de octubre de 2026

Actividad 1 — Proyecto de trabajo

¿Trabajará con un proyecto existente o con uno creado para el laboratorio?

Con un proyecto existente: `demo-academico`, la aplicación Spring Boot que venimos construyendo desde el Lab 02 y que ya dockerizamos en el Trabajo en Clase 03. Usar el proyecto real tiene una ventaja sobre crear uno mínimo desde cero: su *fat jar* pesa 53 MB en lugar de unos pocos megabytes, lo que hace que las diferencias de tamaño entre enfoques de construcción se vean con magnitudes realistas.

¿Qué endpoint simple expone su aplicación?

Se agregó `GET /hola`, que devuelve texto plano y no toca la base de datos. Es deliberado: permite comprobar que el contenedor arrancó y responde sin que el resultado dependa del seeder ni de JPA.

```
@RestController
public class HolaController {
    @GetMapping("/hola")
    public String hola() {
        return "Hola desde Spring Boot con Docker";
    }
}
```

La aplicación conserva además los endpoints del Lab 02 (`/api/estudiantes`, Swagger y la consola H2).

¿Cuál será el nombre base que usará para sus imágenes Docker?

`spring-docker`, con un sufijo por enfoque: `spring-docker-bad`, `spring-docker-good` y `spring-docker-multi`.

Nota sobre dos ajustes al enunciado. El archivo generado por este proyecto se llama `demo-academico-0.0.1-SNAPSHOT.jar` y no `demo-0.0.1-SNAPSHOT.jar`, así que los `COPY` y `CMD` de los tres Dockerfiles se escribieron con ese nombre, siguiendo la recomendación final del enunciado de ser consistente con el jar propio. Y la aplicación se publicó en el puerto **8081** del host, porque el 8080 estaba ocupado por otro contenedor en el equipo; el puerto interno sigue siendo el 8080.

Actividad 2 — Validación local previa

```
$ mvn clean package
[INFO] Compiling 17 source files with javac [debug parameters release 21]
[INFO] Building jar: .../target/demo-academico-0.0.1-SNAPSHOT.jar
[INFO] BUILD SUCCESS

$ ls -lh target/*.jar
53M target/demo-academico-0.0.1-SNAPSHOT.jar

$ java -jar target/demo-academico-0.0.1-SNAPSHOT.jar --server.port=8081
Tomcat started on port 8081 (http) with context path '/'
Started DemoAcademicoApplication in 1.688 seconds

$ curl http://localhost:8081/hola
Hola desde Spring Boot con Docker
HTTP 200
```

¿Por qué es importante validar el proyecto antes de contenerizarlo?

Porque separa dos fuentes de error que, mezcladas, son muy costosas de diagnosticar. Si el contenedor falla y nunca se probó la aplicación por fuera, no hay forma de saber si el problema está en el código, en el Dockerfile, en el mapeo de puertos o en la imagen base; cada hipótesis cuesta un ciclo de construcción. Habiendo validado primero en local, cualquier fallo posterior queda acotado al empaquetado. Es el mismo principio de no introducir dos variables a la vez en un experimento.

¿Qué archivo ejecutable genera Spring Boot después del build?

Un *fat jar* (o *uber jar*) de 53 MB: `target/demo-academico-0.0.1-SNAPSHOT.jar`. El plugin de Spring Boot toma

el jar normal y lo reempaqueta incluyendo todas las dependencias bajo `BOOT-INF/lib` y un Tomcat embebido, de modo que el archivo se ejecuta solo con `java -jar`, sin servidor de aplicaciones externo. Esto es precisamente lo que hace cómodo contenerizarlo: la imagen solo necesita un Java y un archivo.

¿Qué puerto usa la aplicación?

El 8080 dentro del contenedor, que es el valor de `server.port` en `application.yml`. En el host se publicó en el 8081 por el conflicto ya mencionado.

Actividad 3 — Enfoque 1: compilar dentro del Dockerfile

```
FROM maven:3.9-eclipse-temurin-21
WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests
CMD ["java", "-jar", "target/demo-academico-0.0.1-SNAPSHOT.jar"]
```

```
$ du -sh .
53M .                                   ← contexto enviado al daemon

$ docker build -f Dockerfile.bad -t spring-docker-bad .
[INFO] Total time:  53.423 s            ← solo Maven, dentro del contenedor
Successfully tagged spring-docker-bad:latest
real 1:04.50                            ← total del build

$ docker run -d -p 8081:8080 spring-docker-bad
$ curl http://localhost:8081/hola  ->  HTTP 200 "Hola desde Spring Boot con Docker"
```

¿Qué herramientas de build quedan dentro de la imagen?

Se verificó entrando a la imagen en lugar de suponerlo:

```
$ docker run --rm --entrypoint sh spring-docker-bad -c '...'
  mvn:  /usr/bin/mvn
  java: /opt/java/openjdk/bin/java
  /app contiene:
    Dockerfile.bad  Dockerfile.good  Dockerfile.multi
    pom.xml  src  target
  repositorio Maven (~/.m2): 99M
```

Queda **Maven completo** y el JDK con sus herramientas de compilación (`javac`, `jar`, `jlink`). Nada de eso es necesario para ejecutar un `.jar`.

¿Qué archivos adicionales podrían quedar en la imagen final?

El listado de `/app` lo responde: el **código fuente completo** (`src`), el `pom.xml`, los tres Dockerfiles, el directorio `target` con los `.class` y el jar, y un **repositorio Maven de 99 MB** en `/root/.m2` con todas las dependencias descargadas durante la compilación. Que los tres Dockerfiles aparezcan dentro es consecuencia directa de `COPY . .` sin un `.dockerignore`: se copia el directorio entero, incluido lo que no tiene nada que ver con la ejecución.

¿Qué efecto tiene esto sobre el tamaño?

`docker history` muestra exactamente de dónde viene el peso:

```
SIZE      CREATED BY
160MB     /bin/sh -c mvn clean package -DskipTests
55.9MB    /bin/sh -c #(nop) COPY dir:...
```

La imagen base `maven:3.9-eclipse-temurin-21` ya pesa 847 MB; el `COPY . .` añade 56 MB y la compilación otros 160 MB (dependencias descargadas más artefactos intermedios). Resultado: **1.24 GB**, frente a los 661 MB del enfoque optimizado. Casi medio gigabyte de la imagen es herramental que nunca se ejecutará.

¿Por qué este enfoque no es ideal para producción?

Por tres razones de distinta naturaleza. En **transferencia y despliegue**, cada envío al registro y cada descarga en cada nodo mueve el doble de bytes de lo necesario. En **seguridad**, la superficie de ataque crece: la imagen lleva un compilador, un gestor de dependencias y el código fuente; si alguien obtiene ejecución dentro del contenedor, encuentra herramientas para compilar y ejecutar código arbitrario y, además, el fuente para estudiarlo. En **reproducibilidad**, el build descarga dependencias desde internet en cada construcción sin caché, de modo que la imagen depende de que el repositorio remoto esté disponible y devuelva lo mismo.

Actividad 4 — Enfoque 2: compilación separada

```
FROM eclipse-temurin:21-jdk-alpine
WORKDIR /app
COPY target/demo-academico-0.0.1-SNAPSHOT.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
$ docker build -f Dockerfile.good -t spring-docker-good .
Successfully tagged spring-docker-good:latest
real 2.138 s

$ docker run --rm --entrypoint sh spring-docker-good -c '...'
  mvn:  NO
  /app contiene:
    app.jar                              ← solo el jar
  repositorio Maven (~/.m2):             ← no existe

$ curl http://localhost:8081/hola  ->  HTTP 200
```

¿Cuál imagen es más grande?

La del enfoque 1, con diferencia: **1.24 GB frente a 661 MB**. La imagen optimizada ocupa aproximadamente la mitad, y lo hace ejecutando exactamente la misma aplicación.

¿Qué ventaja tiene generar el .jar antes y luego copiarlo?

Que la imagen contiene solo lo que hace falta para ejecutar. El Dockerfile pasa de cinco instrucciones a cuatro y de un paso de compilación a un `COPY`, con dos consecuencias medibles: el tiempo de construcción baja de 64,5 s a **2,1 s**, porque copiar un archivo no se parece en nada a compilar un proyecto; y la imagen base cambia de `maven:3.9-eclipse-temurin-21` (847 MB) a `eclipse-temurin:21-jdk-alpine` (556 MB), que no trae Maven.

¿Qué significa, en este caso, separar build y runtime?

Que la compilación deja de ser responsabilidad del Dockerfile y pasa a ser responsabilidad del proceso que lo precede —el desarrollador o el servidor de integración continua—. Docker recibe un artefacto ya terminado y su único trabajo es definir el entorno donde ese artefacto corre. Son dos preguntas distintas: "cómo se construye esto" y "qué necesita esto para ejecutarse", y mezclarlas en un solo archivo es lo que infla la imagen.

¿Por qué esta separación mejora el despliegue?

Porque reduce lo que viaja y lo que puede fallar. Una imagen de la mitad de tamaño se transfiere en la mitad de tiempo en cada despliegue y en cada réplica; con escalado automático eso se multiplica por el número de instancias. Y un build de 2 segundos en lugar de 64 acorta el ciclo de retroalimentación. Hay además un efecto menos obvio: al compilar fuera, el artefacto que se prueba es exactamente el mismo que se despliega, en lugar de recompilarse dentro de la imagen y arriesgar que el resultado difiera.

Una advertencia sobre este enfoque. Depende de que alguien haya ejecutado `mvn clean package` antes, y nada en el Dockerfile lo garantiza. Si `target/` está vacío, el build falla; peor aún, si contiene un jar viejo, la imagen se construye sin error con código obsoleto. Esta fragilidad es justamente lo que resuelve el enfoque siguiente.

Actividad 5 — Enfoque 3: multi-stage

```
# Etapa 1: build
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests

# Etapa 2: runtime
FROM eclipse-temurin:21-jdk-alpine
WORKDIR /app
COPY --from=build /app/target/demo-academico-0.0.1-SNAPSHOT.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
$ docker build -f Dockerfile.multi -t spring-docker-multi .
real 1.737 s        ← reutilizó la caché de capas del enfoque 1

$ docker build --no-cache -f Dockerfile.multi -t spring-docker-multi .
[INFO] Total time:  49.131 s
real 57.366 s       ← medición honesta, sin caché

$ docker run --rm --entrypoint sh spring-docker-multi -c '...'
  mvn:  NO
  /app contiene:
    app.jar
$ curl http://localhost:8081/hola  ->  HTTP 200
```

La primera medición merece explicación, porque es un resultado real que podría malinterpretarse. El multi-stage construyó en 1,7 s porque su etapa `build` es idéntica a la del enfoque 1 —misma imagen base, mismo `COPY . .`, mismo `RUN mvn clean package`— y Docker reutilizó esas capas ya construidas. Para que la comparación de la Actividad 6 fuera justa se repitió con `--no-cache`, obteniendo 57,4 s. Ambos números son informativos: el primero muestra el valor de la caché en reconstrucciones sucesivas, el segundo el costo real de una construcción desde cero.

¿Qué ocurre en la etapa build?

Se parte de `maven:3.9-eclipse-temurin-21`, se copia el proyecto completo y se ejecuta `mvn clean package -DskipTests`. Al terminar, esa etapa contiene lo mismo que la imagen del enfoque 1: Maven, el fuente, el `.m2` y el `target`.

¿Qué ocurre en la etapa runtime?

Comienza una imagen **nueva** desde `eclipse-temurin:21-jdk-alpine`. El `FROM` descarta todo lo acumulado: lo único que cruza es lo que se copie explícitamente.

¿Qué se copia exactamente de una etapa a la otra?

Un solo archivo, mediante `COPY --from=build /app/target/demo-academico-0.0.1-SNAPSHOT.jar app.jar`. Ni el fuente, ni el `pom.xml`, ni `target/classes`, ni el repositorio Maven.

¿Qué elementos no quedan en la imagen final?

Se verificó dentro de la imagen: no hay `mvn`, `/app` contiene únicamente `app.jar` y no existe `/root/.m2`. Tampoco quedan el código fuente ni los `.class` intermedios. El resultado es **idéntico en contenido y en tamaño (661 MB) al enfoque 2**, pese a que la compilación ocurrió dentro de Docker.

¿Qué ventaja tiene multi-stage frente a compilar y ejecutar en una sola etapa?

Que obtiene las dos cosas que antes eran excluyentes: la imagen pequeña del enfoque 2 y la autosuficiencia del enfoque 1. El build no depende de que alguien haya compilado antes —basta el código fuente y el Dockerfile— y aun así el herramental no llega a producción. Elimina además la fragilidad señalada en la Actividad 4: no puede ocurrir que se empaquete un jar obsoleto, porque el jar se produce en el mismo build.

Actividad 6 — Comparación de enfoques

Criterio	Compila dentro del Dockerfile	Compilación separada	Multi-stage
Tamaño de imagen	1,24 GB	661 MB	661 MB
Tiempo de build (sin caché)	64,5 s	2,1 s	57,4 s
Tiempo de build (con caché)	—	2,1 s	1,7 s
Imagen base	maven:3.9-temurin-21 (847 MB)	temurin:21-jdk-alpine (556 MB)	temurin:21-jdk-alpine (556 MB)
Contenido extra	Maven, src, target, `.m2` (99 MB)	ninguno	ninguno
Tiempo de despliegue	el peor: ~2x bytes por transferencia	el mejor	igual al separado
Seguridad	baja: compilador, fuente y gestor de dependencias en producción	alta: solo JRE/JDK y el jar	alta: igual al separado
Autosuficiencia del build	sí: solo necesita el fuente	no: exige `mvn package` previo	sí: solo necesita el fuente
Adecuación para producción	no recomendable	buena, si hay CI que compile	**la recomendada**

¿Qué sucede con el tamaño de una imagen si incluimos la compilación dentro del Dockerfile?

Prácticamente se duplica: de 661 MB a 1,24 GB en este proyecto. El crecimiento tiene tres fuentes identificadas con `docker history`: la imagen base de Maven es 291 MB mayor que la de ejecución, el `COPY . .` añade 56 MB de contexto y la compilación deja 160 MB entre dependencias descargadas y artefactos intermedios. Nada de eso se ejecuta: es residuo del proceso de construcción que quedó fosilizado en capas de solo lectura.

¿Qué ventajas tiene separar la compilación del proceso de ejecución?

La imagen contiene solo lo necesario para ejecutar, de modo que pesa la mitad, se transfiere más rápido y expone menos. La separación también clarifica responsabilidades: el pipeline de integración continua compila y prueba, y Docker define el entorno de ejecución. Y permite que un fallo de compilación se detecte en el pipeline, donde se dispone de registros, reportes y caché de dependencias, en lugar de dentro de un build de imagen.

¿Cómo impacta esta separación en la seguridad y en el despliegue?

En seguridad, reduce la superficie de ataque de forma concreta: sin `mvn` ni `javac` dentro, un atacante que logre ejecución en el contenedor no tiene herramientas para compilar código, y sin `src` no puede leer la implementación para buscar vulnerabilidades. Menos paquetes significa además menos CVEs que vigilar y parchear. En despliegue, el impacto es cuantitativo: la mitad de bytes por réplica y por nodo, lo que importa especialmente al escalar horizontalmente o al desplegar con frecuencia.

Vale la pena señalar una optimización que estos tres enfoques dejan sobre la mesa: incluso los 661 MB son mejorables, porque `eclipse-temurin:21-jdk-alpine` trae el **JDK** completo y para ejecutar un jar basta un **JRE**. Cambiar a una imagen de solo ejecución, o reducir el runtime con `jlink`, bajaría bastante más el tamaño. El salto de 1,24 GB a 661 MB es el más grande y el más barato, pero no es el límite.

Actividad 7 — Persistencia con volúmenes

```
$ docker volume create datos-spring-lab
datos-spring-lab

$ docker run --rm -v datos-spring-lab:/datos alpine \
      sh -c "echo 'Persistencia Spring Boot' > /datos/info.txt"
  (el contenedor que escribió el archivo fue eliminado por --rm)

$ docker run --rm -v datos-spring-lab:/datos alpine cat /datos/info.txt
Persistencia Spring Boot                ← otro contenedor, el archivo sigue ahí
```

Para que la demostración fuera concluyente se hizo también el caso contrario, escribiendo fuera del volumen:

```
$ docker run --rm alpine sh -c "echo 'sin volumen' > /datos.txt && cat /datos.txt"
sin volumen

$ docker run --rm alpine cat /datos.txt
cat: can't open '/datos.txt': No such file or directory
```

El mismo par de comandos da resultados opuestos según se escriba dentro o fuera del volumen. Esa es la prueba de que el volumen vive fuera del ciclo de vida del contenedor.

¿Qué problema resuelven los volúmenes?

El de que el almacenamiento de un contenedor sea tan efímero como el contenedor mismo. La capa de escritura de un contenedor se crea al arrancarlo y se destruye al eliminarlo; cualquier dato escrito allí desaparece con él. El volumen desacopla el dato del contenedor: lo gestiona el daemon, sobrevive a la eliminación y puede montarse en un contenedor distinto, incluso creado a partir de otra imagen.

¿Qué ocurriría si una base de datos en Docker no usa volumen?

Perdería todos sus datos en cada recreación del contenedor, y recrear contenedores es una operación **rutinaria**, no excepcional: se hace para actualizar la imagen, cambiar una variable de entorno, ajustar un puerto o recuperarse de un fallo. Un `docker compose down` —que es la forma normal de apagar un entorno— bastaría para borrar la base. Dicho de otro modo: sin volumen, operaciones que deberían ser inocuas se vuelven destructivas, y el equipo acaba evitando tocar el contenedor, que es exactamente lo contrario de lo que Docker busca.

¿Por qué este concepto es importante en aplicaciones empresariales?

Porque habilita el principio de infraestructura desechable sobre el que descansan los despliegues modernos. Poder destruir y recrear cualquier contenedor sin consecuencias es lo que hace posibles los despliegues azul-verde, el escalado automático y la recuperación automática ante fallos. Ese principio solo se sostiene si el estado vive fuera: en volúmenes, en servicios gestionados o en almacenamiento externo. La separación entre componentes sin estado —que se replican y se tiran— y estado persistente —que se respalda y se cuida— es una decisión arquitectónica, y el volumen es el mecanismo que la implementa.

Actividad 8 — Redes y comunicación entre contenedores

```
$ docker network create red-spring-lab
7ab8cb2ee9f2945f63a3a304454da75adc15bfb9fc8fd19b9609e64980829078

$ docker run -d --name db-lab --network red-spring-lab \
      -e MYSQL_ROOT_PASSWORD=123456 -e MYSQL_DATABASE=demo mysql:8.0

$ docker ps --filter name=db-lab
NAMES     IMAGE       STATUS          PORTS
db-lab    mysql:8.0   Up 20 seconds   3306/tcp, 33060/tcp     ← sin 0.0.0.0:

$ docker run --rm --network red-spring-lab alpine sh -c 'ping -c 2 db-lab'
PING db-lab (172.24.0.2): 56 data bytes
64 bytes from 172.24.0.2: seq=0 ttl=64 time=0.375 ms
2 packets transmitted, 2 packets received, 0% packet loss
```

Se añadieron dos comprobaciones al guion del enunciado. La primera, el caso contrario, lanzando el mismo ping desde fuera de la red:

```
$ docker run --rm alpine sh -c 'ping -c 1 db-lab'
ping: bad address 'db-lab'
```

La segunda, una conexión real al servicio y no solo un ping, para confirmar que lo que funciona es la comunicación completa y no únicamente la resolución de nombres:

```
$ docker run --rm --network red-spring-lab mysql:8.0 \
      mysql -h db-lab -uroot -p123456 -e 'SHOW DATABASES;'
Database
demo
information_schema
mysql
performance_schema
sys
```

¿Para qué sirven las redes en Docker?

Para crear espacios de comunicación aislados entre contenedores y, dentro de ellos, resolver nombres. Una red definida por el usuario trae un servidor DNS interno que traduce el nombre de cada contenedor a su dirección en esa red: `db-lab` resolvió a `172.24.0.2`. Fuera de la red ese nombre no existe, como demostró el `bad address`. La red define, por tanto, quién puede hablar con quién.

El detalle más instructivo está en la salida de `docker ps`: `db-lab` muestra `3306/tcp` sin ningún `0.0.0.0:` delante, es decir, **no publica ningún puerto al host**, y aun así es plenamente alcanzable desde otro contenedor de la misma red. Esto separa dos conceptos que suelen confundirse: publicar un puerto (`-p`) expone un servicio **hacia el host y hacia el exterior**, mientras que compartir una red lo hace alcanzable **entre contenedores**. Una base de datos de producción normalmente debería estar en el segundo caso y no en el primero.

¿Por qué una aplicación Spring Boot que usa base de datos necesita este concepto?

Porque la URL de conexión tiene que apuntar a algún sitio, y dentro de un contenedor `localhost` se refiere al propio contenedor, no al host ni a la base de datos. Sin una red compartida, la aplicación tendría que usar la dirección IP del contenedor de la base, que Docker asigna dinámicamente y cambia en cada recreación: una configuración así se rompería sola. Con una red común, la URL se escribe `jdbc:mysql://db-lab:3306/demo` y es estable. Esta es, además, la pieza que faltaba en el Trabajo en Clase 03: allí observamos que la aplicación seguía usando H2 porque nunca se le pasó una URL apuntando al servicio `db`.

¿Qué ventaja tiene poder referirse al contenedor por nombre, por ejemplo db-lab?

Que la configuración deja de depender de direcciones asignadas en tiempo de ejecución. El nombre es estable entre recreaciones, mientras que la IP no; se puede destruir y recrear la base de datos y la aplicación la seguirá encontrando. Esto permite versionar la configuración —el mismo `docker-compose.yml` funciona en cualquier máquina— y es la base del descubrimiento de servicios: el nombre lógico se resuelve en el momento de la conexión, de modo que detrás de él puede haber otra instancia, otra IP o incluso varias réplicas.

Actividad 9 — Caso integrador

Una empresa reporta que su aplicación Spring Boot tarda demasiado en desplegarse, sus imágenes pesan mucho, pierde información cuando elimina contenedores y algunos servicios no logran conectarse entre sí.

¿Qué errores identifica en la construcción de sus imágenes?

Los cuatro síntomas apuntan a tres errores concretos. Primero, compilan dentro del Dockerfile en una sola etapa, de modo que las imágenes arrastran el gestor de dependencias, el compilador, el código fuente y el repositorio de artefactos. Segundo, no usan volúmenes para el estado, por lo que los datos viven en la capa de escritura de los contenedores. Tercero, no definen redes propias ni se refieren a los servicios por nombre, probablemente usando `localhost` o direcciones IP que cambian en cada recreación.

¿Qué cambio recomendaría respecto a build y runtime?

Separarlos de forma explícita. La compilación debe producir un artefacto verificado antes de que Docker intervenga, y la imagen final debe partir de una base de ejecución mínima y recibir solo ese artefacto. En la medición de este laboratorio, ese cambio llevó la imagen de 1,24 GB a 661 MB sin tocar una línea de la aplicación.

¿Usaría compilación separada o multi-stage? Justifique.

Multi-stage. Ambas producen la misma imagen de 661 MB, pero se diferencian en la fiabilidad del proceso. La compilación separada depende de que alguien haya ejecutado `mvn package` antes, y si `target/` contiene un jar viejo la imagen se construye **sin ningún error** con código obsoleto: un fallo silencioso, que es la peor clase de fallo. El multi-stage genera el artefacto dentro del mismo build, de forma que la imagen siempre corresponde al código presente. Para una empresa con problemas de despliegue, eliminar un modo de fallo silencioso vale más que los 55 segundos que cuesta el build en frío, sobre todo porque la caché de capas reduce ese costo en las reconstrucciones siguientes: medimos 1,7 s cuando el fuente no cambió.

¿Qué papel cumplen los volúmenes en la solución?

Resuelven directamente la pérdida de información. Todo estado que deba sobrevivir —bases de datos, archivos subidos, caché persistente— debe montarse en volúmenes, no quedar en el contenedor. Con eso, eliminar y recrear contenedores deja de ser una operación destructiva y pasa a ser rutina, que es la condición necesaria para desplegar con frecuencia.

¿Qué papel cumplen las redes en la solución?

Resuelven el problema de conectividad entre servicios. Colocándolos en una red común y refiriéndose a ellos por nombre lógico en lugar de por IP, la configuración se vuelve estable frente a recreaciones. Además permiten restringir la exposición: los servicios internos no necesitan publicar puertos al host, como comprobamos con `db-lab`, lo que reduce la superficie expuesta sin impedir la comunicación interna.

¿Qué mejoraría en tamaño, seguridad, persistencia y despliegue?

En **tamaño**, aproximadamente la mitad de forma inmediata (1,24 GB a 661 MB), con margen adicional si se pasa de JDK a JRE en la imagen de ejecución. En **seguridad**, desaparecen de producción el compilador, Maven y el código fuente, se reduce el número de paquetes sujetos a vulnerabilidades, y los servicios internos dejan de estar publicados al exterior. En **persistencia**, los datos sobreviven a cualquier operación sobre contenedores. En **despliegue**, la mitad de bytes transferidos por réplica, un build de 2 segundos cuando se usa artefacto previo, y la posibilidad real de recrear contenedores sin miedo, que es lo que habilita despliegues frecuentes y recuperación automática.

Conclusión individual

Este laboratorio dejó claro que en Docker el resultado funcional es solo una parte del problema. Los tres enfoques produjeron contenedores que responden idénticamente en `/hola`, y sin embargo uno pesa 1,24 GB y los otros 661 MB: la aplicación es la misma, lo que cambia es qué se arrastró hasta producción. Medir el interior de las imágenes —encontrar Maven, el código fuente y 99 MB de dependencias dentro de la imagen "mala"— fue más revelador que leer la cifra de `docker images`, porque explica el porqué del tamaño y convierte la discusión en un asunto de seguridad y no solo de espacio en disco. El multi-stage resultó ser la opción que no obliga a elegir, porque compila dentro del build sin que el herramental llegue al runtime, y además elimina el riesgo de empaquetar un jar obsoleto en silencio. En volúmenes y redes la lección fue similar y se entendió mejor haciendo el caso contrario: el archivo que desaparece sin volumen y el `bad address` fuera de la red explican más que las pruebas que funcionan. Finalmente, la observación que me pareció más útil enlaza con el trabajo anterior: ver que `db-lab` era alcanzable entre contenedores sin publicar ningún puerto al host separó dos ideas que yo tenía mezcladas, publicar y conectar, y explicó por qué en el Clase 03 la aplicación seguía usando H2 pese a que MySQL estaba arriba. Construir la imagen, gestionar los datos y comunicar los servicios son tres decisiones distintas, y cada una tiene su propio costo si se toma mal.